Parallel Databases: Q & A

1 General Parallelism Concepts

Q1: Parallelism Type Comparison

What form of parallelism (interquery, interoperation, or intraoperation) is likely to be the most important for each of the following tasks?

a. Increasing the throughput of a system with many small queries.

b. Increasing the throughput of a system with a few large queries, when the number of disks and processors is large.

Solution

a. **Inter-query Parallelism:** For many small queries, this provides the best throughput. Parallelizing each small query individually would increase the initiation overhead without significantly reducing response time.

b. **Intra-operation Parallelism:** For a few large queries on a system with many processors and disks, this is essential. Queries typically have few balanceable operations, but each operation must process a massive number of tuples, making intra-operation parallelization the only way to take advantage of the hardware.

2 I/O Parallelism and Partitioning

Q2: Advantages of Horizontal Partitioning

List and explain the three primary horizontal partitioning techniques. Which one is best for point queries?

Solution

The three primary techniques are:

1. **Round-Robin:** Tuples are distributed in a circular fashion across disks. Best for ensuring even load balancing but worst for any form of search (requires scanning all disks).
2. **Hash Partitioning:** A hash function is applied to the partitioning attribute.
3. **Range Partitioning:** Tuples with attribute values in a certain range are stored on the same disk.

Best for Point Queries: Hash Partitioning and **Range Partitioning** are both efficient for point queries because the system can calculate or look up precisely which disk contains the target value, avoiding unnecessary I/O on other disks.

Q3: Range Partitioning Challenges

What is a major disadvantage of range partitioning compared to round-robin?

Solution

The major disadvantage is **Data Skew**. If many tuples fall into a single range (e.g., most employees joined in the year 2023), one disk will be overloaded while others remain idle. Round-robin, by definition, avoids this by distributing tuples equally regardless of their values.

Q4: Range Selection on Range-Partitioned Attributes

In a range selection on a range-partitioned attribute, it is possible that only one disk may need to be accessed. Describe the benefits and drawbacks of this property.

Solution

Benefits:

- If there are few tuples in the queried range, the query is processed quickly on a single disk.
- This reduces the overhead of initiating queries on multiple disks, allowing for more concurrent query execution across the system.

Drawbacks:

- If there are many tuples in the queried range, the query takes longer to execute as there is no *intra-query* parallelism for that specific query.
- Certain disks can become ”hot-spots,” increasing overall system response time.

Optimization: Hybrid Range Partitioning (partitioning small ranges in a round-robin fashion) provides the benefits of range partitioning without its major drawbacks.

Q5: Histogram-based Load Balancing

Recall that histograms are used for constructing load-balanced range partitions.

a. Suppose you have a histogram where values are between 1 and 100, and are partitioned into 10 ranges, 1–10, 11–20, . . . , 91–100, with frequencies 15, 5, 20, 10, 10, 5, 5, 20, 5, and 5, respectively. Give a load-balanced range partitioning function to divide the values into 5 partitions.

b. Write an algorithm for computing a balanced range partition with p partitions, given a histogram of frequency distributions containing n ranges.

Solution

a. **Partitioning Vector: [21, 31, 51, 76]**.

- Total tuples $N = 15 + 5 + 20 + 10 + 10 + 5 + 5 + 20 + 5 + 5 = 100$.
- Target per partition $= 100/5 = 20$.
- Partitions (assuming uniform distribution within ranges): 1–20, 21–30, 31–50, 51–75, and 76–100.

b. **Algorithm:**

- Let h_j be ranges with boundaries $[s_j, e_j]$ and frequencies n_j.
- Total tuples $N = \sum n_j$. Target $k_i = i \cdot (N/p)$ for $i = 1 \ldots p - 1$.
- For each k_i, find range h_j where the k_i-th tuple falls.
- Boundary value $v_i = s_j + \left(\frac{e_j - s_j}{n_j}\right) \cdot \left(k_i - \sum_{l=1}^{j-1} n_l\right)$.

3 Parallel Sorting and Joins

Q6: Parallel External Sort-Merge

Explain why a naive parallel external sort-merge might face performance bottlenecks at the final merging stage.

Solution

While sorting is done in parallel across processors, the final merge stage often requires one processor to receive sorted blocks from all others. If multiple processors send blocks to the same receiver simultaneously, a **communication bottleneck** occurs. **Solution:** Cycle the sorted blocks across processors during the merge phase so that no single processor becomes a dedicated receiver for all blocks at once.

Q7: Fragment-and-Replicate Join

Under what two conditions is Fragment-and-Replicate join used instead of Partitioned Join?

Solution

Fragment-and-Replicate is used when:

1. **Non-Equijoin Conditions:** The join condition is not based on equality (e.g., $r.A > s.B$). Hash-partitioning only works for equality matches.
2. **Extreme Asymmetry:** One relation is very small (e.g., 10MB) while the other is massive (e.g., 500GB). It is cheaper to replicate the small relation to all nodes than to re-partition the massive one.

Q8: Band Join Optimization

Consider join processing using symmetric fragment and replicate with range partitioning. How can you optimize the evaluation if the join condition is of the form $|r.A - s.B| \leq k$, where k is a small constant? (A join with such a condition is called a **band join**).

Solution

Relations r and s are partitioned into n partitions based on ranges: $r_0, \ldots, r_{n-1}$ and $s_0, \ldots, s_{n-1}$.

- **Specialized Replication:** Each partition r_i only needs to be joined with s_{i-1}, s_i, and s_{i+1} (the immediate neighbors), because any tuple beyond those ranges would violate the $|r.A - s.B| \leq k$ constraint.
- **Efficiency:** Each fragment is replicated on only **3 processors** (unlike n processors in the general case).
- **Scalability:** The number of processors required is approximately $3n$ instead of n^2.

Result: For the same hardware, we can use more fragments, making each local join significantly faster due to the high data locality of the band condition.

Q9: Partitioned Parallel Hash Join Phases

Briefly describe the purpose of the two different hash functions (h_1 and h_2) in a parallel hash join.

Solution

- h_1 **(Redistribution):** Used to determine which of the n processors handles a particular tuple. It ensures that tuples from both relations with the same join attribute end up on the same processor.
- h_2 **(Local Build/Probe):** Used locally by each processor. It maps tuples into buckets within the processor's **in-memory hash table** to facilitate high-speed matching.

4 Cost and Interoperation Parallelism

Q10: Parallel Speed-up Estimation

Write the formula for the time taken by a parallel operation. Identify all components.

Solution

The estimated time is:

$$T_{total} = T_{part} + T_{asm} + \max(T_0, T_1, \ldots, T_{n-1})$$

Components:

- T_{part}: Time to partition the relations.
- T_{asm}: Time to assemble the results.
- T_i: Execution time at processor P_i. The total time is dictated by the **slowest** processor ($\max(T_i)$).

Q11: Pipelined vs. Independent Parallelism

How do pipelined parallelism and independent parallelism differ in their data dependency?

Solution

- **Pipelined Parallelism:** Involves a **direct dependency**. One operation produces tuples that are consumed by the next operation in a chain. They run simultaneously using a producer-consumer model.
- **Independent Parallelism:** Involves **no dependency**. Two different operations (e.g., two different joins in a query tree) are executed at the same time on different clusters of processors because they don't need each other's output.

Q12: Clustering Operations on Processors

With pipelined parallelism, it is often a good idea to perform several operations in a pipeline on a single processor, even when many processors are available.

a. Explain why.

b. Would the arguments you advanced in part (a) hold if the machine has a shared-memory architecture? Explain why or why not.

c. Would the arguments in part (a) hold with independent parallelism? (i.e., cases where it is better to perform several independent operations on one processor).

Solution

a. **Data Transfer Overhead:** The speed-up from parallelizing operations can be offset by the overhead of transferring data between processors. If the computation cost is low compared to the communication cost, clustering operators on a single processor is more efficient.

b. **Shared-Memory Architecture:** The argument does **not** hold as strongly here. Transferring tuples via shared memory is extremely fast and efficient, so the communication overhead is minimal compared to shared-nothing systems.

c. **Independent Parallelism:** Yes, the argument holds. If two independent operations supply their output to a common third operator, running all three on the same processor may be faster than shipping tuples across the network to a central assembly node.

5 Query Optimization

Q13: Teradata vs. Volcano Heuristics

Compare the heuristic used by Teradata with the Volcano model for parallel query optimization.

Solution

- **Teradata Heuristic:** It assumes total parallelism for every operation. It does not use pipelining and parallelizes every operation across **all** available processors. This makes the search for a plan similar to sequential optimization.
- **Volcano Model:** It first chooses the most efficient **sequential plan** and then parallelizes it by inserting **Exchange Operators**. These operators encapsulate the logic for data movement and partitioning.

Q14: The Exchange Operator

What is the primary benefit of the ”Exchange Operator” introduced in the Volcano model?

Solution

The Exchange Operator allows the database engine to **reuse existing sequential implementations** of operators (Join, Sort, etc.). Because the Exchange Operator handles all the ”parallelism logic” (shuffling, partitioning, networking), the other operators can remain ”unaware” they are running in a parallel cluster.

6 Availability and Resilience

Q15: High Availability and System Resilience

Large-scale parallel database systems store an extra copy of each data item on disks attached to a different processor, to avoid loss of data if one of the processors fails.

a. Instead of keeping the extra copy of data items from a processor at a single backup processor, it is a good idea to partition the copies across multiple processors. Explain why.

b. Explain how virtual-processor partitioning can be used to efficiently implement this.

c. What are the benefits and drawbacks of using RAID storage instead of storing an extra copy of each data item?

Solution

a. **Load Balancing during Failure:** If all replicas of processor P_i are stored on a single backup P_j, then if P_i fails, P_j must handle its own load plus 100% of P_i's load, becoming a bottleneck. By partitioning (skewing) the replicas across n other processors, each only takes on $1/n$ of the extra load.

b. **Virtual-Processor Partitioning:** The system maps data to a large number of **virtual processors** rather than directly to physical ones. These virtual processors are distributed among the physical nodes. If a physical node fails, its virtual processors are simply "migrated" to several different surviving physical nodes.

c. **RAID vs. Mirroring:**

- **RAID Benefit:** More storage-efficient (parity-based redundancy like RAID 5 saves space compared to 100% mirroring). It handles disk-level failure transparently to the DB.
- **RAID Drawback:** RAID has a "write penalty" (parity calculations). Software mirroring allows the DBMS to use the second copy to speed up **read queries** (load balancing), which is harder to optimize with hardware RAID.

Q16: Indexing Partitioned Relations

Suppose we wish to index a large relation that is partitioned. Can the idea of partitioning (including virtual processor partitioning) be applied to indices? Explain your answer, considering cases where:

a. The index is on the partitioning attribute of the relation.

b. The index is on an attribute other than the partitioning attribute.

Solution

Yes, the concept of partitioning can be applied to indices just as it is to data relations.

a. **Index on Partitioning Attribute:** This is the most efficient scenario. We can use **local indices** where each processor builds an index for its own fragment of data. Queries on this attribute are routed directly to the specific processor(s) containing the relevant range, making lookups extremely fast and scalable.

b. **Index on Non-Partitioning Attribute:**

- **Global Partitioned Index:** The index is partitioned across nodes independently of how the data is partitioned. This allows search queries to be routed to a single index node, but updates become expensive as they may involve multiple nodes (data node and index node).
- **Local Index:** Every node stores an index for only its own local data. While updates are cheap and local, search queries must be sent to **all processors** (broadcast), which increases system-wide overhead for large clusters.

Q17: Dynamic Repartitioning and Load Balancing

Suppose a well-balanced range-partitioning vector becomes unbalanced due to updates.

a. Suppose a virtual processor has a significant excess of tuples. Explain how repartitioning can be done by splitting the partition.

b. If virtual partitions are allocated to processors via a mapping table, explain how load can be balanced when a Particular node has excess load.

c. Does query processing have to be stopped during these operations? Explain.

Solution

a. **Partition Splitting:** A single virtual processor (range) is split into two or more smaller ranges. New virtual processors are created for these ranges. This increases the total number of virtual processors in the system, allowing the data to be redistributed more granularly.

b. **Migration via Mapping Table:** If a physical node is overloaded, one or more of its **virtual processors** can be migrated to a different physical node with lower utilization. We simply update the mapping table to point the virtual processor ID to the new physical network address. Only the data for the specific virtual processor needs to be moved.

c. **Online Operations:** No, query processing does **not** have to be stopped. Repartitioning and migration can be done **online**. While the data is being moved/split, the old mapping remains active. Once the data transfer is complete, the mapping table is updated atomically, and subsequent queries are routed to the new location.

Q18: Partitioning Selection per Query Type

For each of the three partitioning techniques (round-robin, hash, range), give an example of a query for which that technique would provide the fastest response.

Solution

1. **Round-Robin:** A query that requires a full scan of the entire relation (e.g., `SELECT COUNT(*) FROM r`). Round-robin ensures the scan is perfectly distributed across all disks with no skew.
2. **Hash Partitioning:** A point query on the partitioning attribute (e.g., `SELECT * FROM r WHERE r.A = 100`). The system hashes 100 to find the exact disk, avoiding $n - 1$ other disks.
3. **Range Partitioning:** A range query on the partitioning attribute (e.g., `SELECT * FROM r WHERE r.A BETWEEN 10 AND 20`). The system only accesses disks corresponding to that specific range.

Q19: Skew in Partitioning

What factors result in skew when a relation is partitioned by: (a) Hash partitioning, (b) Range partitioning? What can be done to reduce it?

Solution

- **Hash Partitioning Skew:** Caused by many tuples having the **same value** for the partitioning attribute (value skew). *Correction:* Choose a better attribute with more distinct values or a more uniform distribution.
- **Range Partitioning Skew:** Caused by either **value skew** or an **improper partitioning vector** where some ranges have significantly more tuples than others. *Correction:* Use a **histogram** to create load-balanced ranges or use virtual processor partitioning.

Q20: Partitioned Parallelism for Non-Equijoins

Give an example of a join that is not a simple equi-join for which partitioned parallelism can still be used. What attributes should be used for partitioning?

Solution

A **Band Join** (e.g., $r.A \bowtie s.B$ where $|r.A - s.B| \leq k$) or a join with a range condition ($r.A \leq s.B$). **Partitioning:** Use **Range Partitioning** on attributes $r.A$ and $s.B$. This preserves the sort order and allows each processor to join its local fragment with only its immediate neighbors' fragments (as seen in Q7).

Q21: Parallelizing Relational and Aggregation Operations

Describe a good way to parallelize each of the following:

a. The difference operation ($r - s$)

b. Aggregation by the `COUNT` operation

c. Aggregation by the `COUNT DISTINCT` operation

d. Aggregation by the `AVG` operation

e. Left outer join, if the join condition involves only equality

f. Left outer join, if the join condition involves comparisons other than equality

g. Full outer join, if the join condition involves comparisons other than equality

Solution

a. **Difference ($r - s$):** Hash-partition both relations on all attributes using the same function. Each processor computes the local difference.

b. **COUNT Aggregation:** Each processor computes a local count; the global count is obtained by summing all local counts.

c. **COUNT DISTINCT Aggregation:** Hash-partition on the attribute being counted. Each processor computes the distinct count for its partition; results are summed.

d. **AVG Aggregation:** Each processor computes local `SUM` and `COUNT`. The final average is computed from global sums and counts.

e. **Left Outer Join (Equality):** Hash-partition both relations on the join attribute and perform local joins. Unmatched left tuples are padded with `NULL`s.

f. **Left Outer Join (Non-Equality):** Use fragment-and-replicate. Replicate the right relation and join locally; unmatched left tuples are padded with `NULL`s.

g. **Full Outer Join (Non-Equality):** Use fragment-and-replicate and track unmatched tuples from both relations, padding them with `NULL`s.

Q22: Pipelined Parallelism Trade-offs

Describe the benefits and drawbacks of pipelined parallelism.

Solution

Benefits:

- **Reduced Intermediate Storage:** Tuples are passed directly between operators, avoiding the need to write large intermediate relations to disk.
- **Lower Latency:** The first results can be returned to the user almost immediately after the pipeline starts, even before the entire query is finished.

Drawbacks:

- **Chain Length Limitation:** Relational queries often have few operations, so the "depth" of the pipeline is small.
- **Blocking Operators:** Certain operators (Sort, Hash Join build phase) must see the entire input before producing output, breaking the pipeline.
- **Communication Costs:** Shipping every tuple across the network can be expensive.

Q23: Handling Transaction-Heavy Workloads

Suppose you handle a workload of many small transactions using shared-nothing parallelism.

a. Is intraquery parallelism required? What form of parallelism is appropriate?

b. What form of skew is significant in this scenario?

c. If transactions access an *account* record and its associated *account type* master record, how would you partition/replicate data?

Solution

a. **No, intraquery parallelism is not required.** Each transaction is small and only accesses a few records. Instead, **inter-query parallelism** is used to execute many independent transactions on different processors simultaneously.

b. **Execution Skew:** If most transactions access the same physical records (e.g., a few "hot" accounts), the node holding those records will be overloaded while others are idle.

c. **Partitioning and Replication:**

   - **Account Table:** Partition it across nodes based on `account_id` to distribute the primary transaction load.
   - **Account Type Master:** Since it is rarely updated and small, **replicate** it on every node. This allows transactions to perform the join locally on any node without performing any network I/O.

Q24: Partitioning Attribute Selection

The attribute on which a relation is partitioned has a significant impact on cost.

a. What attributes would be candidates for partitioning?

b. How would you choose between alternative techniques based on workload?

c. Is it possible to partition on more than one attribute? Explain.

Solution

a. **Candidate Attributes:** Attributes frequently used in **equality predicates** (perfect for hash), **range predicates** (perfect for range), or as **join attributes**.

b. **Decision Criteria:**

- If the workload has many **point lookups**: Choose **Hash Partitioning**.
- If the workload has many **range queries**: Choose **Range Partitioning**.
- If we want maximum **load balance** for scans: Choose **Round-Robin**.

c. **Multi-attribute Partitioning:** Yes. This is called **Multi-dimensional Partitioning** or **Grid Files**. For example, you can partition by `Branch` as the primary dimension and `Date` as the secondary, ensuring data is distributed across nodes while still being clustered logically for complex queries.

Mastering these concepts is essential for designing high-performance distributed and parallel data systems.